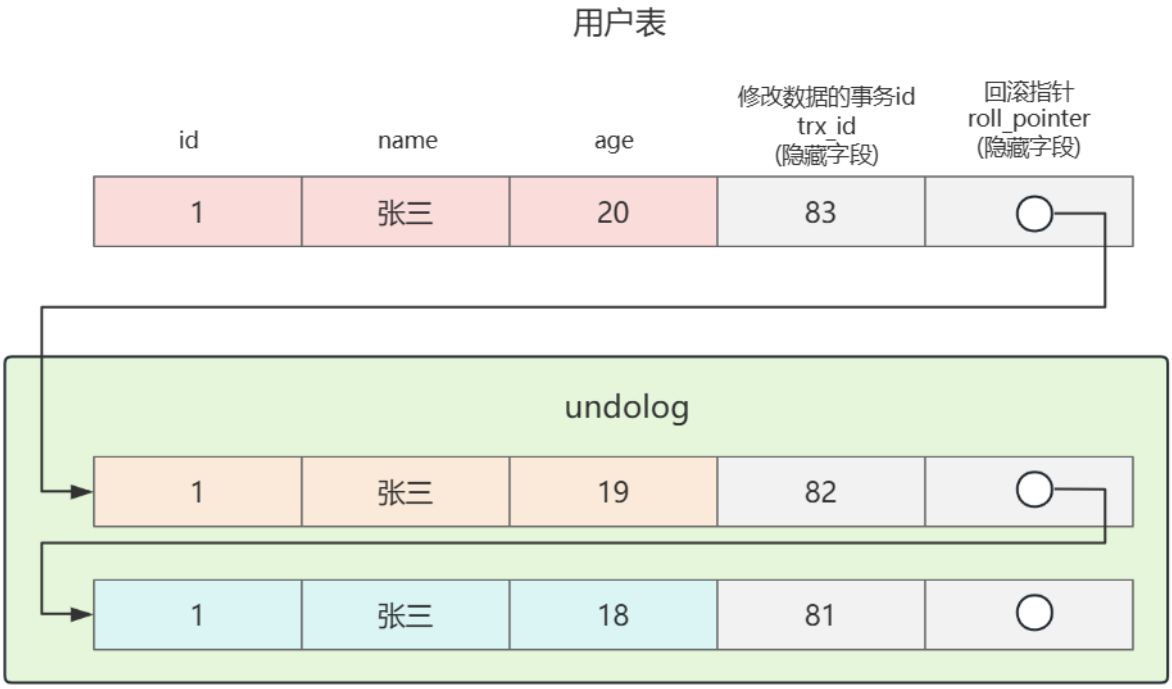
日志

undo log

undo log的基本作用

mysql有事务, 事务的特性之一就是原子性, 执行一系列sql, 要么全成功要么全失败, 后面的sql失败了, 已经执行的也要回滚. 那么问题来了, 我数据都修改了, 怎么回滚到没有修改的状态呢?

这个时候就需要保存数据的不同版本了, 每修改一次, 就把原始的数据版本保存下来. 把数据的不同版本通过一个回滚指针串连起来, 就叫undo log版本链.



也就说, 事务执行中, 修改了数据, MySQL会把旧数据记录到undo log日志里. 如果事务正常提交, 那就结束. 如果事务因为异常, 或者执行了手动回滚操作, 那就通过undo log找到旧版本的数据, 回滚. 这也就是为什么说, undo log可以保证事务的原子性.

undo log和mvcc

undo log第二个作用就是配合readview和表的隐藏字段实现MVCC. 就是说MVCC执行普通select语句, 就会对比事务id和readview来查看这个数据版本对当前事务的是否可见, 不可见的话, 就继续沿着undo log版本链查找当前事务可见的数据版本. 这个在MVCC那个文件已经详细讲过了.

redo log

什么是buffer pool

大家都知道MySQL的数据是存在磁盘上的, 但是每次从磁盘中读数据, 写数据是不是太慢了, 怎么办呢?

你想想, 写业务代码的时候, 查数据太慢了怎么办? 加一层缓存.

你再想想, CPU执行速度那么快, 内存速度又比较慢, 怎么办? CPU三级缓存.

所以遇事不决, 就加层缓存. 那你猜猜MySQL怎么做的?

它也是这么想的. MySQL设计了一个缓冲池, 叫buffer pool, MySQL存数据是以页为单位的, 一个页16kb, 每查询一条数据, 会从硬盘把一页的数据加载出来, 把这个数据页放到 `Buffer Pool` 中.

为什么要一次把一个页的数据放入buffer pool

那么问题就来了, 查一条数据, 把这一条数据放缓存中就行了, 不至于把一个数据页都取出来放到buffer pool吧. 这是什么操作?

幼儿园老师讲操作系统的时候应该教过, 程序有空间局部性. 就是说, 一个内存位置被访问了, 这就意味着未来很可能附近的内存位置还会被访问, 因为代码中会大量使用数组这种连续结构, 数组的第一个位置被访问了, 第二位置第三个位置大概率也会被访问的. 同样的, 你写个管理系统, 里面分页展示了很多数据, 你看完一个页的数据, 还会接着点下一页点下一页, 你刷抖音, 刷小红书, 一直往下滑, 这也是连续翻页的过程.

MySQL一次把一个页的数据放入buffer pool中, 那对于取连续数据来说, 命中率就很高了, 这就是利用了程序的空间局部性原理. 后续取数据先从buffer pool中取, 没有就去磁盘中读. 写数据也是先写入buffer pool, 但是不会立刻就刷入磁盘, 而是修改的页标记为赃页, 然后由后台线程在某个时间把赃页刷盘.

buffer pool未及时刷盘, 数据丢失了怎么办? 为什么要有redo log

那么问题来了, buffer pool是在内存中的, 没有立即刷盘, 那MySQL要是挂了, 数据不就丢了.

所以就有了redo log，mysql每次把某个磁盘页修改的数据记录在redo log中，然后事务提交之后就刷盘redo log.

如果buffer pool中的数据未刷盘就宕机了，那么就可以读取redo log来恢复数据. 这也就是为什么说redo log能保证事务的持久性，让mysql能够做到崩溃回复.

为何不刷盘buffer pool, 而是刷盘redo log?

那为什么修改buffer pool之后不直接刷盘呢，而去写redo log，去刷盘redo log?

因为buffer pool刷盘是随机io，刷盘的时候需要找到某个磁盘页，然后修改，然后再去找另外一个磁盘页，再修改，本来磁盘操作就慢，随机io的话就更慢了.

但是redo log只记录在那个磁盘位置做了怎样的修改，刷盘时，只需要往redo log日志文件后追加就行了，这就是顺序io，不需要找这个磁盘页找那个磁盘页，磁盘的磁头只需要沿着一个方向移动就行了. 这也就是为什么redo log里面要记录数据页的物理修改, 而不是直接记录数据.

所以只要redo log一刷盘, 即便MySQL崩溃了也能恢复, 事务的持久性也就能得到保证.

redo log没刷盘, MySQL就崩溃了, 会不会丢失数据?

那么问题来了, redo log没刷盘, MySQL就崩溃了, 怎么办?

解答这个问题之前, 必须得先说一下redo log其实也不是直接写入磁盘的, redo log也有自己的缓存, 叫redo log buffer. redo log会先写入到redo log buffer, 然后统一把redo log buffer中的数据刷盘到磁盘中.

MySQL有个参数叫innodb_flush_log_at_trx_commit, 这个参数就是控制redo log刷盘时间的.

- 如果值为0, 事务提交时会把redo log的数据放入redo log buffer, 并不会刷入磁盘
- 如果值为1, 事务提交时会把redo log的数据刷入磁盘, 刷盘完成后才会告诉客户端事务执行成功了. 这也就能保证事务只要完成, 即便MySQL崩溃数据也不会丢失. 所以一般情况下, 把这个参数设置为1, 那事务提交redo log就能刷盘, 事务的持久性也能得到保证. 1也是这个参数的默认值.

- 如果值为2, 事务提交时会把redo log的数据写入操作系统文件缓存中, 也就是写入page Cache. 操作系统本身对文件也是有缓存的, 这个缓存就是page cache. 数据写入page cache后, 操作系统就会在某个时间真正把数据写入磁盘. 那只要操作系统不崩溃, 那就能保证持久性. 但是如果说电脑突然断电了, 那数据就丢了.

到这儿你就会发现, 计算机就是个巨大的缓存, 处处都是缓存, 一层一层的缓存.

binlog

为什么需要binlog?

现在思考这样一个场景, 你要做数据库备份, 要做备份恢复, redo log能用来做备份恢复吗? 或者要做主从, 主从复制能用redo log吗? 或者说, 你删库跑路了, 老板能用redo log恢复数据吗?

不行, 为啥呢? redo log不是一提交事务就刷盘数据吗, 咋就不能做备份恢复或者主从复制呢?

- redo log是循环写, 日志空间大小是固定的, 全部写满就从头再开始写, 边写边擦除, 它只能记录事务提交后没有被刷盘的数据, 已经刷盘的会从redo log中擦除掉.
- redo log是事务级的数据记录, 不是数据库级的, 它只能做哪些因为断电或者数据库故障导致buffer pool中的数据未刷盘的数据恢复.
- 不能做数据库全量数据恢复.

那咋办呢? 要做备份恢复或者主从同步应该用啥呢?

只要对MySQL做了变更, 无论是数据增删改, 还是表结构增删改, 都会记录一个日志, 叫binlog.

- redo log是循环写, binlog是追加写, 它记录的是全量日志. 所以binlog更适合用来做备份恢复, 主从同步. 所以你删库后最好把binlog也删了再跑路.
- redo log是物理日志, 记录内容是“在某个数据页上做了什么修改”
- binlog 是逻辑日志, 记录的内容就类似于sql语句本身. 所以binlog非常适合做备份恢复和主从同步.

redo log cache

之前聊了buffer pool缓存, 聊了redo log的redo log buffer缓存, 那binlog有没有类似的缓存机制的, 有的有的. 缓存虽迟但到.

binlog也有对应的binlog cache, 事务执行过程中, 会先把binlog日志写到binlog cache, 事务提交的时候, 就会把binlog cache刷到磁盘中.

MySQL日志设计, 你能学到什么?

那么问题来了, 通过MySQL日志的设计, 你学到了什么?

- 要保证持久化, 就得记日志.
- 对于连续写的日志, 最好搞一个cache, 先写cache, 然后再异步刷盘. 通过日志就能做备份恢复, 主从同步.

说到这儿, 你能想到了什么? 好像redis的rdb aof也是这么个逻辑.

两阶段提交

事务提交后, redolog和binlog都要刷盘, 但是如果一个刷盘成功了, 一个失败了. 两份日志就不一致了, 怎么办?

为了解决两份日志之间的一致性问题, MySQL将`redo log`的写入拆成了两个步骤 `prepare` 和 `commit`，这就是幼儿园老师常说的两阶段提交. 整个执行流程分了这么几步:

- 开始事务
- 更新数据
- 写入redo log (此时redo log是prepare阶段)
- 提交事务, 写入binlog, redolog 设置为commit阶段

如果写入redo log异常了, 也就是prepare异常了, 会发生什么?

- 那redo log写入异常, 那redo log和binlog都没有数据, 那就直接回滚事务.

如果写入binlog异常会发生什么?

- 写入binlog异常, 那么mysql根据redo log进行日志数据恢复时, 会发现redo log处于prepare阶段，并且没有对应binlog日志, 那就回滚事务.

如果redo log设置commit异常了, 会发生什么?

- 那么mysql就发现redo log为prepare阶段, 但是能找到对应的binlog日志
- 那redo log和binlog是一致的, MySQL也会认为数据是完整的, 直接提交事务.

所以两阶段提交最终还是要看binlog, 只要binlog刷盘了, 那就能提交事务.